

KỊCH BẢN THUYẾT TRÌNH — ASSIGNMENT 06

Mạng RNN hồi quy · Bộ slide: CHUYEN_DE_RNN_A06.pptx (9 slide) · Thời lượng ~8 phút

Người trình bày: Giáp Minh Hiếu — MSSV: B23DCCN299 — Lớp: D23CTPM01-B — GVHD: TS. Trần Đình Quế

Nguyên tắc: mỗi slide chỉ nói 1 ý chính bằng lời đơn giản; code trên slide chỉ CHỈ VÀO giải thích vài dòng then chốt, không đọc lại cả khối. Công thức thì ĐỌC BẰNG LỜI (ví dụ “ h_t bằng $\tanh$ của W nhân x_t cộng U nhân h_{t-1} trừ 1 cộng bias”) rồi mới chỉ số.

MỞ ĐẦU — trước khi chuyển slide 1 (~30 giây)

Chào thầy và các bạn. Em là Giáp Minh Hiếu, mã sinh viên B23DCCN299. Hôm nay em trình bày Assignment 06 về mạng RNN — mạng nơ-ron có trí nhớ, dùng để dự đoán dữ liệu theo thời gian. Em sẽ đi theo 9 phần: vì sao cần RNN, ý tưởng và hàm `rnn_cell`, công thức với ba ma trận W U V , một ví dụ tính tay, bốn hàm tiền xử lý dữ liệu, công thức lan truyền ngược BPTT, từng dòng của vòng lặp huấn luyện, áp dụng vào hai tập dữ liệu thật, và cuối cùng là hạn chế cùng LSTM GRU.

SLIDE 1 — Vì sao cần mạng RNN? (~45 giây)

Dữ liệu chuỗi là dữ liệu có thứ tự theo thời gian — ví dụ giá vàng và giá bạc theo ngày từ 1968 đến 2021, hoặc giá cổ phiếu Amazon kèm khối lượng giao dịch từ 1997 đến 2023 — hai tập dữ liệu Kaggle em dùng trong bài.

Vấn đề: mạng nơ-ron thông thường nhìn mỗi đầu vào như một ảnh chụp riêng lẻ — nó không biết hôm qua diễn ra gì. Mà giá ngày mai lại phụ thuộc rất nhiều vào mấy hôm trước. Vì vậy mình cần một mạng CÓ TRÍ NHỚ — nhớ quá khứ, cộng thông tin hôm nay, để đoán ngày mai. Đó chính là RNN.

Chỉ vào khối code bên phải — hàm `make_windows`: nó cắt chuỗi thành các cửa sổ 30 ngày liên tiếp. Dòng `np.stack` gom mọi dải 30 ngày thành ma trận X hình N phần 30 phần 2; dòng dưới tính nhãn y là hiệu giá ngày kế trừ giá hôm nay — tức biết 30 ngày, đoán biến thiên của ngày mai. Đây là code thật của notebook 03.

SLIDE 2 — Ý tưởng: trạng thái ẩn h_t là “trí nhớ” (~60 giây)

Trọng tâm của RNN là vector trạng thái ẩn h_t — hiểu đơn giản là trí nhớ: nó tóm tắt mọi thứ mạng đã thấy tới thời điểm t .

Mỗi bước mạng chỉ làm ba việc: đọc đầu vào mới x_t , mở trí nhớ cũ h_{t-1} trừ 1, gộp lại thành trí nhớ mới h_t . Quan trọng: cùng MỘT bộ trọng số lặp qua từng ngày — chuỗi dài bao nhiêu tham số vẫn nguyên.

Chỉ vào code numpy thuần của notebook 00: hàm `rnn_cell` chính là MỘT bước. Dòng `a` bằng `W` nhân `x-t` cộng `U` nhân `h-prev` cộng `bias` — trong đó `W` nhân `x-t` là đọc tín hiệu mới, `U` nhân `h-prev` là mở trí nhớ hôm qua. Return `tanh` của `a` — hàm `tanh` nén về khoảng trừ 1 cộng 1 giữ số ổn định. Vòng `for` bên dưới lặp đúng cell này qua từng ngày; sau cùng nhân `V` đọc trí nhớ cuối ra dự đoán. Hình bên trái: các ô RNN nối nhau là CÙNG một ô lặp lại, mũi tên ngang là trí nhớ truyền qua.

SLIDE 3 — Công thức & 3 ma trận `W`, `U`, `V` (~60 giây)

Công thức trung tâm của cả bài, đọc bằng lời: `h-t` bằng `tanh` của `W` nhân `x-t` cộng `U` nhân `h-t` trừ 1 cộng `b-h`; nhân `V` cộng `b-y` ra kết quả `y-t`.

Ba ma trận như ba nhân vật, xem bảng: `W` kích thước hidden nhân input, đọc thông tin mới; `U` hidden nhân hidden — trái tim của RNN — giữ trí nhớ chuyển từ hôm qua sang hôm nay; `V` đọc trí nhớ thành kết quả. Vì cả ba dùng chung cho mọi thời điểm nên số tham số KHÔNG phụ thuộc độ dài chuỗi.

Chỉ vào code phải: bên PyTorch, `W` là `weight_ih`, `U` là `weight_hh`, `V` là tầng `Linear` cuối. Hai dòng dưới: `rnn` of `Xb` truyền trọn chuỗi ra `out` hình batch phần 30 phần 64, nhưng chỉ `out` dấu hai chấm trừ một — đọc NGÀY CUỐI — vì dự báo ngày mai chỉ cần trí nhớ mới nhất. Bên Keras: `W` gọi `kernel`, `U` gọi `recurrent`. Hai notebook của em verify ba trọng số này khớp numpy tới sai số 10 mũ trừ 16.

SLIDE 4 — Tính thử một bước (~55 giây)

Để không tưởng tượng công thức suông, em tính tay một bước với ma trận nhỏ 2 nhân 2, seed cố định.

Bốn dòng bảng là bốn phép tính: một — `W` nhân `x-1` với `x-1` bằng 1 và 0,5; hai — `U` nhân `h-0`, mà `h-0` khởi tạo bằng 0 nên bước này ra 0; ba — cộng hai kết quả, nén `tanh` được trí nhớ `h-1`; bốn — `V` nhân `h-1` ra dự đoán `y-1` dương 0,662.

Sang ngày thứ hai lặp `y` hết: thay `x-2`, lấy `h-1` vừa tính làm trí nhớ cũ. Code bên phải sinh đúng các số trong bảng — ai cũng chạy lại kiểm tra được. Nhớ quy ước tên: PyTorch `weight_ih` là `W`, Keras `kernel` là `W`.

SLIDE 5 — Bốn hàm tiền xử lý của pipeline (~70 giây)

Slide này đi qua TỪNG HÀM tiền xử lý — bốn khối bên trái ứng bốn nhóm lệnh bên phải.

Một — chia `chronological 70 15 15`: cắt theo THỜI GIAN, không xáo trộn — nếu xáo trộn, mẫu tương lai lọt vào tập train gây rò rỉ dữ liệu. Hai — `MinMaxScaler`: `z` bằng `x` trừ `min` chia `max` trừ `min`, đưa về khoảng 0-1; bắt buộc CHỈ fit tập train — fit cả chuỗi là lộ `min` `max` của tương lai cho mô hình. Ba — `make_windows`: trượt cửa sổ

30 ngày, nhãn là ĐỀ-LTA vàng scaled — giá mai trừ giá nay. Bốn — guard clip cộng trừ 3 sigma: sigma là độ lệch chuẩn của ĐỀ-LTA train; nhãn ngoài 3 sigma bị kéo về biên, vì lợi suất có đuôi dày — vài ngày khủng hoảng ĐỀ-LTA cực lớn sẽ chi phối hàm loss nếu không chặn.

Chỉ vào code phải: bốn nhóm lệnh đánh số đúng bốn khối trái — đây là code thật notebook 01, dòng cuối np.clip chính là guard vừa nói.

SLIDE 6 — BPTT: công thức lan truyền ngược qua thời gian (~70 giây)

Mạng học bằng BPTT — backpropagation through time. Đọc công thức bằng lời, khối trái từ trên xuống.

Dòng đầu: loss L bằng trung bình bình phương sai số. Dòng hai: đạo hàm L theo V dễ — sai số nhân thẳng h-t. Dòng ba là nhân tử then chốt: đạo hàm h-i theo h i trừ 1 bằng U chuyển vị nhân đường chéo của 1 trừ tanh bình phương — nhân tử JACOBIAN này lặp lại mỗi bước lùi. Dòng tư: đạo hàm theo W là tổng kép — mỗi thời điểm k CỘNG DỒN từ mọi đường ngược về nó, mỗi đường là một tích các nhân tử vừa nói.

Hệ quả ở hai dòng cuối: chuẩn của tích bị chặn bởi chuẩn U mũ t trừ k. Chuẩn U dưới 1 — cấp số nhân về 0, gradient BIẾN MẤT; trên 1 — cấp số nhân ra vô cùng, BÙNG NỔ. Đồ thị phải từ notebook 00 minh họa. Chữa bùng nổ: cắt gradient — khối dưới phải — g nhân min của 1 và c chia chuẩn g: giữ hướng, chặn độ lớn. Biến mất thì phải chữa bằng kiến trúc — LSTM, slide cuối.

SLIDE 7 — Vòng lặp huấn luyện từng dòng (~60 giây)

Đây là vòng lặp huấn luyện THẬT của notebook 01, em đánh số 1 đến 8, bảng phải giải thích từng lệnh.

Một — lấy batch 64 cửa sổ. Hai — zero_grad xoá gradient vòng trước để không cộng dồn. Ba — forward: model of xb chạy rnn_cell 30 bước rồi Linear ra ĐỀ-LTA mũ, tính loss MSE. Bốn — loss dot backward: PyTorch tự chạy BPTT tính đạo hàm theo W U V. Năm — opt điểm step: Adam cập nhật, mỗi tham số có tốc độ học riêng. Sáu — đo loss trên tập val dưới torch no grad cho nhanh. Bảy — nếu val không giảm bốn epoch liền thì EarlyStopping dừng — chống quá khớp. Tám — phục hồi bản trọng số TỐT NHẤT, không phải bản cuối.

Nếu được hỏi thêm: state_dict là chụp toàn bộ trọng số hiện tại; patience bằng 4 là số epoch khoan dung. Keras làm y hệt bằng callback EarlyStopping với restore best weights.

SLIDE 8 — Áp dụng: dự báo Δ residual trên 2 tập thật (~60 giây)

Áp dụng lên cả hai tập. Mẹo then chốt ở khối code trái: em dự báo BIẾN THIÊN giá — ĐỀ-LTA — chứ không dự báo mức giá. Dự báo bằng mức giá cho RMSE thua naive 2 đến 5 lần, vì tập train giá thấp, tập test giá cao, mạng bị kéo về trung bình.

Pipeline: model xuất ĐỀ-LTA mũ scaled, chặn trong cùng biên cộng trừ 3 sigma; rồi cộng vào giá hôm nay của cửa sổ — z -prev — được giá mai scaled; hàm `inverse_gold` đổi về USD: nó đem cột bạc bằng 0 rồi inverse transform lấy cột vàng, vì scaler fit trên hai cột.

Kết quả bảng phải: giá vàng RMSE 15,45 so naive 15,48; Amazon 3,13 so 3,14 — RNN thắng baseline naive “ngày mai giống hôm nay” trên cả hai tập, R bình phương trên 0,99, MAPE 0,6 đến 1,8 phần trăm. Kỹ thuật ĐỀ-LTA nhất quán ở cả bốn notebook.

SLIDE 9 — Hạn chế & phát triển: LSTM, GRU (~55 giây)

RNN đơn giản hay quên quá khứ xa — gốc rễ ở slide 6: gradient chỉ đi qua phép NHÂN U lặp lại nên biến mất. LSTM thêm BĂNG CHUYỀN trí nhớ dài hạn c -t: gradient đi qua phép CỘNG và cổng quên nên gần như nguyên vẹn.

Công thức trái, đọc từng dòng: f -t là cổng QUÊN — giữ lại bao nhiêu ký ức cũ; i -t cổng GHI — nhận bao nhiêu thông tin mới; c -tilde là nội dung mới đề xuất; dòng c -t bằng f nhân c cũ cộng i nhân c tilde chính là băng chuyền; o -t cổng XUẤT lộ bao nhiêu; h -t bằng o nhân $\tanh c$ -t. Mỗi cổng là hàm sigma đầu ra 0 đến 1 — như vận van.

GRU bên phải là bản gọn: gộp quên và ghi thành hai cổng z và r , không có c -t riêng, ít tham số khoảng 25 phần trăm nhưng hiệu quả tương đương. Dưới cùng: với PyTorch chỉ đổi MỘT dòng — `nn.RNN` sang `nn.LSTM` hoặc `nn.GRU` — notebook 00 của em cài và so sánh cả ba.

KẾT (~20 giây — không có slide)

Tóm lại: RNN là mạng có trí nhớ với ba ma trận W U V dùng chung mọi thời điểm; em đã hiểu nó từ numpy thuần qua BPTT tới PyTorch và Keras, trang bị pipeline bốn hàm tiền xử lý, và dự báo tốt trên hai tập dữ liệu thật. Trên đó là toàn bộ bài trình bày của em, em xin phép nhận câu hỏi từ thầy và các bạn.

Phụ lục — Q&A dự phòng

Câu hỏi dự phòng	Trả lời ngắn
Vì sao chọn dự đoán biến thiên giá thay vì mức giá?	Mức giá giữa các giai đoạn lệch nhau quá lớn (train giá thấp, test giá cao) nên mạng bị kéo về trung bình — RMSE thua baseline naive 2 đến 5

	lần. Dự đoán biến thiên Δ rồi cộng vào giá hôm nay thì phân phối ổn định hơn hẳn: 4 notebook đều đạt $R^2 \geq 0.986$, 3/4 thắng naive.
Trong BPTT, vì sao gradient biến mất còn bùng nổ thì cắt gradient được?	Biến mất và bùng nổ cùng một cơ chế — tích Jacobian chứa U mũ số bước: chuẩn U dưới 1 thì tích về 0, trên 1 thì tích ra vô cùng. Clipping chỉ co gradient về ngưỡng khi nó LỚN — tức trị được bùng nổ; còn khi gradient đã gần 0 thì không có gì để “kéo giãn” lại, phải đổi kiến trúc (LSTM/GRU) để gradient đi qua phép cộng thay vì phép nhân.
LSTM chống biến mất bằng cơ chế nào cụ thể?	Đường băng c_t : $c_t = f_t \odot c_{(t-1)} + i_t \odot \tilde{c}_t$. Đạo hàm của PHÉP CỘNG theo $c_{(t-1)}$ là f_t — một đại lượng học được gần 1 — thay vì tích các khối U chuyển vị nhân đường chéo tanh. Gradient theo thời gian nhân thêm f_t mỗi bước (có thể giữ ≈ 1) thay vì bị nhân shrinking liên tục.
Sao không dùng LSTM luôn cho đẹp số?	Assignment này tập trung hiểu RNN gốc — kiến trúc, W U V và BPTT, nên em giữ vanilla RNN làm chính. Notebook 00 vẫn cài và so sánh LSTM/GRU ở phần phát triển; đổi sang LSTM chỉ là 1 dòng.
Directional Accuracy chỉ ~51–53%, có phải mô hình yếu?	Đúng như kỳ vọng của thị trường hiệu quả: hướng đi ngày mai gần như ngẫu nhiên từ cửa sổ 30 ngày. Điều em nhấn mạnh là RMSE/MAPE thắng baseline, còn hướng đi chỉ nhỉnh hơn tung xu một chút — em nêu trung thực trong notebook.
Vì sao chia dữ liệu theo thời gian mà không xáo trộn?	Nếu xáo trộn, mẫu tương lai lọt vào tập train → rò rỉ thông tin (data leakage), kết quả ảo. Chuỗi thời gian phải chia chronological 70/15/15 và scaler chỉ fit trên train.